PoC-SQL-Schema Master-MD: Evidence Store (Kap. 5.4)

Datum: 2026-03-13 **Zweck:** DDL-Komponenten → DP-Mapping → Move-Zuordnung → APA7-Belegstellen **Input:** v01.sql (minimal), v02_enterprise.sql (enterprise), evidence_store_architektur.md **Ziel:** Konkrete Schema-Referenzen für thesis-writer bei "GO"

---

1. Schema-Evolution: v01 → v02 (DSR Design Rationale)

Aspekt	v01 (Minimal)	v02 (Enterprise)	DSR-Begründung
Schema-Separation	Kein Schema	`medical.*` / `compliance.*`	DP4 Governance-Trennung: Payload ≠ Telemetrie
Rollen	Keine	3 Rollen (ingest/auditor/admin)	DP5 Least Privilege
RLS	Nein	Row-Level Security + Policies	DP1 + DP5 Privacy by Design
Hash-Chain	`hash_value`/`previous_hash` (Spalten, keine Logik)	Trigger-Funktion `set_hash_chain()` mit SHA-256	DP5.3 Tamper Evidence
Immutability	Trigger `trg_prevent_delete_update()`	Trigger `prevent_update_delete()` (identisch)	DP2 Append-Only
Indexes	3 (reporting, checked_at, failures)	4 + Materialized View	DP5.2 Performance SLO
Privacy View	`vw_quality_gate_reporting` (Basis)	View + Materialized View `mv_auditor_daily`	DP1 + DP4
Payload-Referenz	`evidence_blob_url TEXT`	`payload_id UUID REFERENCES medical.payload_objects`	R3 FK-Constraint statt URL-String

Design-Iteration: v01→v02 = DSR Build-Evaluate Micro-Cycle (Hevner et al., 2004)

---

2. DDL-Komponenten → Design-Prinzipien → Moves

K1: Schema-Separation (`medical.*` / `compliance.*`)

```
CREATE SCHEMA IF NOT EXISTS medical;
CREATE SCHEMA IF NOT EXISTS compliance;
```

Dimension	Wert
Design-Prinzip	DP4 (Governance-Trennung), R3 (Healthcare-Anforderung)
Move	**M1** (Motivation: Payload/Telemetrie-Trennung)
Claim	DSGVO Art. 9 erfordert strikte Trennung medizinischer Payload von regulatorischer Telemetrie
Belegstellen	Nweke & Yeng (2026, PDF S. 3–4): privacy-preserving compliance; Kholkar & Ahuja (2025, S. 1): "data minimization"; Golpayegani et al. (2024): metadata separation
DSR-Eigenleistung	**E2**: Architektur-Pattern (encrypted Blob vs. SQL) in keiner Quelle als dediziertes Pattern
SQL-Artefakt	`medical.payload_objects` (Blob-Metadaten) + `compliance.quality_gate_results` (Telemetrie)

K2: Least-Privilege RBAC (3 Rollen + RLS)

```
CREATE ROLE app_ingest_role NOINHERIT;    -- CI/CD Pipeline: INSERT only
CREATE ROLE auditor_role NOINHERIT;        -- Auditoren: SELECT only
CREATE ROLE admin_compliance_role NOINHERIT; -- Compliance Officer: ALL
```

```
-- RLS Policies
CREATE POLICY pol_insert_ingest  ... FOR INSERT TO app_ingest_role;
CREATE POLICY pol_select_auditor ... FOR SELECT TO auditor_role;
CREATE POLICY pol_all_admin      ... FOR ALL    TO admin_compliance_role;
```

Dimension	Wert
Design-Prinzip	DP5 (Cloud-native Integrierbarkeit)
Move	**M4** (Least-Privilege RBAC)
Claim	DB-native Rollen-Enforcement statt Application-Layer Access Control
Belegstellen	Kholkar & Ahuja (2025, S. 1): "enforce least privilege and data minimization"; Burns et al. (2025, PDF S. 8–9): real-world RBAC in EU AI Act context; Eisenberg (2025, PDF S. 12): UCF Access Controls

Dimension	Wert
DSR-Eigenleistung	Nicht primär — gut belegt (Q4: 3 explizit + 5 implizit)
SQL-Artefakt	3 Rollen + 3 RLS-Policies + REVOKE/GRANT-Kette

K3: quality_gate_results Table (Kern-Schema)

```
CREATE TABLE compliance.quality_gate_results (
    audit_id      BIGSERIAL PRIMARY KEY,
    model_name    TEXT NOT NULL,
    model_version TEXT NOT NULL,
    pipeline_id   TEXT NOT NULL,
    run_id        UUID NOT NULL,
    gate_type     TEXT NOT NULL CHECK (gate_type IN
('Strategisch','Technisch','Compliance')),
    decision      TEXT NOT NULL CHECK (decision IN ('PASS','FAIL')),
    gate_name     TEXT NOT NULL,
    policy_version TEXT NOT NULL,
    payload_id    UUID NOT NULL REFERENCES
medical.payload_objects(payload_id),
    checked_at    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    inserted_by   TEXT NOT NULL DEFAULT current_user,
    hash_value    TEXT NOT NULL,
    previous_hash TEXT,
    notes         TEXT
);
```

Dimension	Wert
Design-Prinzip	DP1 (Compliance-Lifecycle), DP2 (Traceability)
Move	**M2** (Schema als DSR-Artefakt) + **M6** (quality_gate_results + CAC/AAC)
Claim	Jede Gate-Evaluation = 1 Evidence Record. CDV-Framework-Output als strukturierter Record.
Belegstellen	Butt (2026, PDF S. 1–2): "signed, content-addressed artifacts into a tamper-evident Evidence Backbone"; Muhammad et al. (2026, PDF S. 1): "versioned policy specification and executable checks for evidence artifacts"; Eisenberg (2025, PDF S. 8): unified schema concept
Feld-Mapping	

Feld	Herkunft	DP	Quelle
`gate_type` CHECK	3-Säulen-Taxonomie aus Kap. 5.2	DP1	Nweke & Yeng (2026): Clause-to-Control

Feld	Herkunft	DP	Quelle
`decision` CHECK	PASS/FAIL Gate-Semantik	DP2	Muhammad et al. (2026, S. 5): "gate engine that checks inputs"
`policy_version`	Versionierte Policy-Referenz	DP2	Muhammad et al. (2026, S. 5): "policy is versioned"
`payload_id` FK	R3 Trennung → FK statt Blob-URL	DP4, R3	DSR-Eigenleistung E2
`hash_value` / `previous_hash`	Hash-Chain-Felder	DP5.3	Butt (2026, S. 8): "content addressed using SHA-256"
`inserted_by`	Audit-Trail: wer hat eingefügt	DP5	Kholkar & Ahuja (2025, S. 1): audit logging

DSR-Eigenleistung E4: Synthese aus 9 Quellen zu vollständigem Schema mit DP-Mapping. Keine einzelne Quelle liefert ein vergleichbares Schema.

K4: Immutability-Trigger (Append-Only Enforcement)

```
CREATE OR REPLACE FUNCTION compliance.prevent_update_delete()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'EU AI Act compliance: UPDATE/DELETE on audit evidence
is prohibited';
END; $$;

CREATE TRIGGER trg_prevent_ud
BEFORE UPDATE OR DELETE ON compliance.quality_gate_results
FOR EACH ROW EXECUTE FUNCTION compliance.prevent_update_delete();
```

Dimension	Wert
Design-Prinzip	DP2 (Traceability), DP5.3 (Tamper Evidence)
Move	**M3** (Immutability-Trigger)
Claim	SQL-Level Enforcement: Append-Only als technische Garantie, nicht als Konvention
Belegstellen	Kholkar & Ahuja (2025, S. 1): "complete provenance, traceability, and audit logging"; Butt (2026, S. 1): "tamper-evident Evidence Backbone"; Muhammad et al. (2026, S. 8): "immutable evidence/decision trail"; Burns et al. (2025, PDF S. 5): immutability requirement
DSR-Eigenleistung	Teil von E1 (zusammen mit Hash-Chain) — SQL-Trigger-Enforcement ist Designbeitrag

K5: Hash-Chain Trigger-Funktion (SHA-256 Verkettung)

```sql
CREATE OR REPLACE FUNCTION compliance.set_hash_chain()
RETURNS trigger LANGUAGE plpgsql AS $$
DECLARE
    prev_hash_local TEXT;
    payload TEXT;
BEGIN
    SELECT q.hash_value INTO prev_hash_local
      FROM compliance.quality_gate_results q
     ORDER BY q.audit_id DESC LIMIT 1;
    NEW.previous_hash := prev_hash_local;
    payload := concat_ws('|',
        coalesce(NEW.model_name, ''),
        coalesce(NEW.model_version, ''),
        coalesce(NEW.pipeline_id, ''),
        coalesce(NEW.run_id::text, ''),
        coalesce(NEW.gate_type, ''),
        coalesce(NEW.decision, ''),
        coalesce(NEW.gate_name, ''),
        coalesce(NEW.policy_version, ''),
        coalesce(NEW.payload_id::text, ''),
        coalesce(NEW.checked_at::text, ''),
        coalesce(NEW.inserted_by, ''),
        coalesce(NEW.previous_hash, '')
    );
    NEW.hash_value := encode(digest(payload, 'sha256'), 'hex');
    RETURN NEW;
END; $$;

CREATE TRIGGER trg_set_hash_chain
BEFORE INSERT ON compliance.quality_gate_results
FOR EACH ROW EXECUTE FUNCTION compliance.set_hash_chain();
```

Dimension	Wert
Design-Prinzip	**DP5.3** (Tamper Evidence — Sub-Extension von DP5)
Move	**M7** (Hash-Chain-Integrität) ⚡ **Primäre DSR-Eigenleistung**
Claim	Verkettete SHA-256-Hashes über alle Evidence Records. Nachträgliche Manipulation bricht Kette → O(1) Tamper Detection. Art. 12 EU AI Act Logging-Pflicht.
Belegstellen	Butt (2026, S. 8): "content addressed using SHA-256. Each pointer carries a URI, digest, and signer identity"; Joseph (2023): Merkle-Tree-basierte Audit-Trails mit <5ms Verifikationslatenz (⚠ Venue-Caveat: arXiv-Preprint)
DSR-Eigenleistung	**E1: Hash-Chain-Immutability** — Q2 Matrix: 1/9 explizit. Kein Paper beschreibt SQL-Level Hash-Chain für AI-Governance Evidence Stores.
Abgrenzung	≠ Blockchain (kein Konsens-Mechanismus), ≠ Merkle Tree (lineare Kette vs. Baumstruktur). Lightweight Variante für DB-Trigger-Kontext.

Dimension	Wert
Mechanik	`prev_hash = SELECT hash_value ORDER BY audit_id DESC LIMIT 1` → `new_hash = SHA-256(alle Felder ‖ prev_hash)`

K6: Composite Indexes + Performance SLO

```
CREATE INDEX idx_qgr_reporting       ON compliance.quality_gate_results
(model_name, gate_type);
CREATE INDEX idx_qgr_checked_at      ON compliance.quality_gate_results
(checked_at);
CREATE INDEX idx_qgr_failures_partial ON compliance.quality_gate_results
(checked_at) WHERE decision = 'FAIL';
CREATE INDEX idx_qgr_run_id          ON compliance.quality_gate_results
(run_id);
```

Dimension	Wert
Design-Prinzip	**DP5.2** (Performance — Sub-Extension von DP5)
Move	**M5** (Composite Indexes + 100ms SLO)
Claim	Query < 100ms SLO für Audit-Reporting über Millionen Gate-Durchläufe. Partial Index auf `decision = 'FAIL'` für Incident-Response.
Belegstellen	Eisenberg (2025, PDF S. 8): unified schema enables efficient querying; Muhammad et al. (2026, S. 5): "bounded evidence schema, which produces versioned JSON artifacts"; Butt (2026, S. 9): gate schema applied across 5 gate types
DSR-Eigenleistung	**E3**: Keine Quelle definiert quantitative Performance-Ziele für Audit-Query-Response
Index-Strategie	

Index	Zweck	Query-Pattern
`idx_qgr_reporting`	Compliance-Dashboard	`WHERE model_name = ? AND gate_type = ?`
`idx_qgr_checked_at`	Zeitreihen-Analyse	`WHERE checked_at BETWEEN ? AND ?`
`idx_qgr_failures_partial`	Incident-Response (Partial)	`WHERE decision = 'FAIL' AND checked_at > ?`
`idx_qgr_run_id`	Pipeline-Run-Lookup	`WHERE run_id = ?`

K7: Privacy-Safe Reporting Views

```sql
CREATE OR REPLACE VIEW compliance.vw_quality_gate_reporting AS
SELECT audit_id, model_name, model_version, pipeline_id, run_id,
       gate_type, gate_name, decision, checked_at, hash_value,
previous_hash
FROM compliance.quality_gate_results;
-- Kein: notes, inserted_by, payload_id → PII-Minimierung

CREATE MATERIALIZED VIEW compliance.mv_auditor_daily AS
SELECT date_trunc('day', checked_at) AS day_bucket,
       model_name, gate_type, decision, count(*) AS run_count
FROM compliance.quality_gate_results
GROUP BY 1,2,3,4;
```

Dimension	Wert
Design-Prinzip	DP1 (Compliance-Lifecycle), DP4 (Governance-Trennung)
Move	**M1** (Privacy-Kontext) + **M4** (Auditoren-Zugriff via Views)
Claim	Kein direkter Tabellenzugriff für Auditoren. View maskiert PII-nahe Felder. Materialized View für Dashboard-Performance.
Belegstellen	Kholkar & Ahuja (2025, S. 1): "data minimization"; Nweke & Yeng (2026): privacy-preserving compliance checks
Masking-Logik	View exkludiert: `notes` (Freitext), `inserted_by` (Benutzername), `payload_id` (FK zu med. Daten)

3. Traceability-Kette: R → DP → Gate → Evidence (M8)

```
Requirement (Kap. 4)
    → Design Principle (Kap. 5.1)
        → Gate-Instanz (Kap. 5.2, CDV-Framework)
            → Evidence Record (quality_gate_results, Kap. 5.4)
                → Hash-Chain-Verkettung (DP5.3)
                    → Conformity Bundle (Audit-Report)
```

Belegstellen	
Nweke & Yeng (2026, PDF S. 5–6)	"Clause-to-Control mapping" für Traceability
Muhammad et al. (2026, S. 1)	"traceability and explainability" als Kernprinzip
Kholkar & Ahuja (2025, S. 1)	"complete provenance, traceability, and audit logging"
Butt (2026, S. 1–2)	"Clause-to-Artifact Traceability (C2AT)"

Evidenzstärke: 🟢 Q5 PERFEKT — 9/9 Quellen adressieren Traceability

4. CAC/AAC-Zuordnung im Schema (M6)

Schema-Komponente	CAC (Compliance-as-Code)	AAC (Audit-as-Code)
`gate_type` CHECK	Strategisch/Technisch/Compliance → CAC-Taxonomie	—
`policy_version`	Versionierte Policy-Referenz → CAC	—
Immutability-Trigger	—	Beweissicherung → AAC
Hash-Chain-Trigger	—	Nachweiskette → AAC
`decision` PASS/FAIL	Gate-Entscheidung → CAC	Persistiert als Evidence → AAC
Privacy Views	—	Audit-Zugriff → AAC

Quelle: Muhammad et al. (2026): "Audit-as-Code" als Framework-Name; Nweke & Yeng (2026): Compliance-as-Code Operationalisierung

---

5. DSR-Eigenleistungen → SQL-Komponenten

#	Eigenleistung	SQL-Komponente	DP	Move	Lücken-Begründung (aus Evidenz-Matrix)
E1	Hash-Chain-Immutability	`set_hash_chain()` Trigger + `prevent_update_delete()` Trigger	DP5.3 + DP2	M7 + M3	Q2: 1/9 explizit. Keine SQL-Level Hash-Chain für AI-Governance.
E2	Payload/Telemetrie-Trennung	`medical.*` / `compliance.*` Schema-Separation + FK-Constraint	DP4 + R3	M1	Q3: 3 explizit, 4 implizit. Kein dediziertes Architektur-Pattern.
E3	Performance SLO 100ms	4 Composite/Partial Indexes + Materialized View	DP5.2	M5	Keine quantitativen Performance-Ziele in Literatur.
E4	PostgreSQL Evidence Store Schema	Gesamtes v02-Schema (7 DDL-Blöcke, 5 Komponenten)	Alle DP	M2	Synthese: Keine Quelle liefert vollständiges Schema.

---

6. Quellen-Referenz-Index (für Writer)

Quelle	Zotero-Key	PDF-Seiten-Bereich	Kern-Beitrag für 5.4	In Moves

Quelle	Zotero-Key	PDF-Seiten-Bereich	Kern-Beitrag für 5.4	In Moves
Butt (2026)	V6HKHA5B	S. 1–2 (Evidence Backbone), S. 8 (SHA-256, content-addressed), S. 9 (Gate Schema), S. 10 (immutable waivers), S. 31 (append-only)	Primärquelle: content-addressed, tamper-evident, C2AT, Conformity Bundle	M2,M3,M5,M6,M7,M9
Muhammad et al. (2026)	IZVYTSTV	S. 1 (Audit-as-Code Framework), S. 5 (gate engine, versioned policy, hash), S. 8 (immutable trail), S. 13 (DV/MV/PL/AT/RR Dimensionen)	Primärquelle: AAC namensgebend, readiness score, evidence bundle	M2,M5,M6,M8,M9
Kholkar & Ahuja (2025)	VLMNBUST	S. 1 (least privilege, provenance, traceability, audit logging, data minimization)	Primärquelle: Policy-as-Prompt, Least Privilege, Provenance	M1,M3,M4,M8
Nweke & Yeng (2026)	XCM4Q2WP	S. 3–6 (Clause-to-Control, privacy-preserving)	Stützquelle: CAC-Operationalisierung, Traceability	M1,M8
Eisenberg (2025)	JUK36XAW	S. 8 (unified schema), S. 12 (UCF Access Controls)	Stützquelle: UCF, Schema-Konzept	M2,M4,M5,M6
Burns et al. (2025)	2GGF93BE	S. 5 (immutability), S. 8–9 (RBAC, EU AI Act)	Stützquelle: Dynamo real-world, Compliance	M4,M9
Joseph (2023)	Elicit	—	Supplementary: Merkle Trees, <5ms Latenz, Art. 12. ⚠ Venue-Caveat	M7

7. Negativ-Checklist (Schema-bezogen)

- ❌ **Keine vollständigen DDL-Listings im Fließtext** — nur konzeptuelle Beschreibung + Verweis auf PoC (Kap. 6.3)
- ❌ Keine `CREATE TABLE`-Syntax im Kapiteltext — Schema-Logik verbal beschreiben
- ❌ Keine Azure-spezifischen Konfigurationen — gehören in Kap. 5.5 (PoC)
- ❌ Keine Wiederholung der Gate-Taxonomie — Verweis auf Kap. 5.2
- ❌ `pgcrypto` Extension nur erwähnen, nicht erklären (PostgreSQL-Standardbibliothek)

---

8. Writer-Anweisungen

Schema-Referenzierung im Fließtext:

```
Pattern: "Der Evidence Store realisiert [Prinzip] durch [Mechanismus]
         (vgl. Schema-Komponente K[x] in Abschnitt 6.3)."
```

Kein Code im Fließtext, aber:

- Tabellen-/Feldnamen in `monospace` erlaubt
- Trigger-Konzept verbal beschreiben: "Ein BEFORE-Trigger auf INSERT-Operationen berechnet..."
- Hash-Mechanik verbal: "...verkettete SHA-256-Hashes, bei denen jeder neue Record den Hash-Wert seines Vorgängers einschließt"

Forward-References:

- Schema-DDL → Kap. 6.3 (PoC-Walkthrough)
- Azure PostgreSQL Flexible Server → Kap. 5.5 (Cloud-Deployment)
- Gate-Instanzen → Kap. 5.2 (CDV-Framework)
- Decision Logs → Kap. 5.3 (Policy Engine)